Lab 4 Report

Name.Malik Musawar Ali Roll num.su92-bssem-s25-006 section . BSSE 4A

1. Problem and Assumptions

In this lab I used A* search and UCS on a small campus graph. The start is S and the goal is G. I calculated g, h and f by hand for the first three expansions, compared A* with UCS, and then changed one heuristic value to an overestimate to see what happens.

The graph is undirected and the edge weights are my own assumptions. S-A 2, S-B 4, A-C 2, A-D 4, B-D 1, C-G 5 and D-G 5. The successor order is S has A, B. A has S, C, D. B has S, D. C has A, G. D has A, B, G. G has C, D.

The heuristic values are S 8, A 6, B 6, C 4, D 5 and G 0. These values are also my assumption.

Tie breaking. The node with the smaller f is taken first. If f is equal, the entry that was added to the queue first is taken first. Counting rule. A node is counted when it is taken from the queue and processed, and the goal is counted too. A stale entry is a node that is already processed. It is skipped and not counted. I used the same rule for A* and UCS. UCS is the same code with h equal to 0 for every node.

2. Manual g, h and f Calculation

Expansion 1

Selected node is S. g(S) = 0, h(S) = 8, f(S) = 0 + 8 = 8. S is the start so it is the only node in the frontier.

Successors in order. A has g = 0 + 2 = 2, h = 6, f = 2 + 6 = 8. B has g = 0 + 4 = 4, h = 6, f = 4 + 6 = 10.

Remaining frontier is A (f 8) and B (f 10). A is selected next because 8 is smaller than 10.

Expansion 2

Selected node is A. g(A) = 2, h(A) = 6, f(A) = 2 + 6 = 8.

Successors in order. S is already processed so it is skipped. C has g = 2 + 2 = 4, h = 4, f = 4 + 4 = 8. D has g = 2 + 4 = 6, h = 5, f = 6 + 5 = 11.

Remaining frontier is C (f 8), B (f 10) and D (f 11). C is selected next because 8 is the smallest f.

Expansion 3

Selected node is C. g(C) = 4, h(C) = 4, f(C) = 4 + 4 = 8.

Successors in order. A is already processed so it is skipped. G has g = 4 + 5 = 9, h = 0, f = 9 + 0 = 9.

Remaining frontier is G (f 9), B (f 10) and D (f 11). G is selected next because 9 is the smallest f. When G is processed the search stops. I ran the code and its g, h, f values and frontiers match my hand calculation.

3. Heuristic Audit

I found the true remaining cost to G for every node and compared it with h. S has h 8 and true cost 9. A has 6 and 7. B has 6 and 6. C has 4 and 5. D has 5 and 5. G has 0 and 0. No h is bigger than the true cost, so the heuristic is admissible. I also checked every edge and h(n) is never bigger than the edge weight plus h of the next node, so the heuristic is consistent too.

4. A* and UCS Comparison

A* with the original heuristic

- Returned path S-A-C-G with cost 9.
- Independent sum of the weights is 2 + 2 + 5 = 9.
- Processing order S, A, C, G. Expansion count is 4. Stale entries skipped 0.

UCS (zero heuristic)

- Returned path S-A-C-G with cost 9.
- Independent sum of the weights is 2 + 2 + 5 = 9.
- Processing order S, A, B, C, D, G. Expansion count is 6. Stale entries skipped 1.

Both found the same optimal path with the same cost. A* expanded 4 nodes and UCS expanded 6, because the heuristic guided A* away from B and D. In UCS the node D was added to the queue two times, with g 6 from A and with g 5 from B. The entry with g 5 came out first. The old entry with g 6 came out later, and it was skipped as a stale entry without counting it. A* stopped before this happened. The goal is counted in both runs, so the counts are fair. If the goal was not counted, the numbers would be 3 and 5.

5. Overestimate Experiment

I changed only one value, h(C). The original value is 4 and the modified value is 10. The true remaining cost of C is 5 (the edge C-G), so 10 is an overestimate by 5.

Result after the change. A* returned S-B-D-G with cost 10 (independent sum 4 + 1 + 5 = 10). The processing order was S, A, B, D, G and the expansion count was 5. Before the change it was S-A-C-G with cost 9 and 4 expansions. So the path, the cost and the count all changed, and the new answer is not optimal.

Why it happened. After A is processed, C has f = 4 + 10 = 14, which is more than the other entries (B has 10 and D has 11). So C was left in the queue, and G was reached through B and D with f = 10 before C came out. The good path through C was never tried.

Why an overestimate removes the guarantee. A* is optimal only if h never overestimates, because then f of a node on the best path is never bigger than the real cost of that path, and it will not be pushed behind a worse goal. With an overestimate, a node on the best path can look too expensive and can be ignored for too long.

Extra run. I also tried h(C) = 6, which overestimates by only 1. This time A* still returned S-A-C-G with cost 9, but it expanded 5 nodes instead of 4 because B was expanded too. A run can look unchanged in path and cost, but the guarantee is still gone, because on another graph the same small error could change the answer.

6. Comparison of the Alternatives

UCS only uses g, so it is always optimal but it expands more nodes. A* with an admissible and consistent heuristic is also optimal and expands fewer nodes here. A* with an overestimating heuristic can be faster to give an answer, but the answer may be wrong. In this lab UCS expanded 6 nodes, A* expanded 4, and A* with the overestimate expanded 5 and gave a worse path.

7. Failure / Edge Case

Input. A* from S to G with the modified heuristic h(C) = 10.

Observed behaviour. A* returned S-B-D-G with cost 10, but the optimal cost is 9.

Likely cause. h(C) is bigger than the true remaining cost 5, so the heuristic is not admissible and f(C) = 14 pushed the best path to the back of the queue. It also broke consistency on the edges C to A and C to G.

Proposed improvement. Run the heuristic audit before A*, with the true costs, and fix any value that is bigger than the true cost, for example set h(C) back to 4 or to 5. If the heuristic cannot be trusted, use UCS.

8. Limitations

I used one small graph with 6 nodes, made up weights and made up heuristic values. The expansion counts depend on the tie breaking and on the counting rule. I did not measure running time. On a bigger graph the difference between A* and UCS can be much larger.

9. Reflection and Responsible Practice

[Write your reflection here in your own words. For example, what was difficult in the hand calculation and what you learned about admissible heuristics.]

- All data is made up. No real private information or passwords are uploaded to GitHub.
- The graph, the weights and the heuristic values are written as assumptions so they are not mistaken for real data.
- In a real system such as a route planner, a wrong heuristic can give a worse route without any error message, so the heuristic should be checked.
- No outside code was copied. The ideas are from the references below.